

Material para a formación profesional inicial

Familia profesional	IFC	Informática e comunicacións
Ciclo formativo	CSIFC03	Desenvolvemento de aplicacións web
Grao		Superior
Módulo profesional	MP0613	Desenvolvemento web en contorno servidor
Unidade didáctica	UD05	Formularios e persistencia da información
Actividade	A01	Persistencia da información
Autores		Margarita Pin Rodríguez, Víctor M. Lourido Estévez
Nome do arquivo		CSIFC03_MP0613_V000502_UD05_A02_Persistencia_info rmación

© 2014 Xunta de Galicia.

Consellería de Cultura, Educación e Ordenación Universitaria.

Este traballo foi realizado durante unha licenza de formación retribuída pola Consellería de Cultura, Educación e Ordenación Universitaria e ten licenza Creative Commons BY-NC-SA (recoñecemento - non comercial - compartir igual). Para ver unha copia desta licenza, visitar a ligazón <http://creativecommons.org/licenses/by-nc-sa/3.0/es/>.

Índice

1.	A2. Persistencia da información	3
1.1	Introdución	3
1.2	Actividade	3
1.2.1	Cookies	3
	Finalidade das <i>cookies</i>	3
	Cookies e seguridade	3
	Transmisión das <i>cookies</i>	6
	Envío dunha cookie ao navegador	6
	Recuperación das cookies almacenadas no navegador	7
	Emprego de <i>cookies</i> en PHP	8
	Almacenamento de información nunha cookie	8
	Recuperación da información almacenada nunha cookie	9
	Forzar o borrado dunha <i>cookie</i>	9
	Almacenamento de arrays nas cookies	9
1.2.2	Sesións	10
	Mantemento da información dos usuarios	10
	Configuración das sesións	12
	Xestión da información das sesións	13
	Peché manual da sesión	14
	Almacenamento da información da sesión	15
	Secuestro e fixación da sesión	15
	Secuestro da sesión	15
	Fixación da sesión	16
1.2.3	Autenticación de usuarios	17
	As contrasinais	17
	O salt	17
	Mantemento e peche da sesión do usuario	17
	Establecer un tempo límite de inactividade	18
	Manter aberta a sesión aínda que se peche o navegador do usuario	18
1.3	Tarefas	21
1.3.1	Tarefa 1. Almacenamento de información nas <i>cookies</i>	21
	Autoavaliación	22
1.3.2	Tarefa 2. Almacenamento de información na sesión do usuario	27
	Autoavaliación	29
1.3.3	Tarefa 3. Autenticación de usuarios	31
	Autoavaliación	35
2.	Materiais	43
2.1	Textos de apoio ou de referencia	43
2.2	Recursos didácticos	43
3.	TA1. Cálculo do hash en PHP	44
3.1	Funcións e extensións para o cálculo do hash	44

1. A2. Persistencia da información

1.1 Introducción

Na actividade que nos ocupa aprenderanse os seguintes conceptos e manexo de destrezas:

- Xestionar as *cookies* almacenadas nos navegadores dos usuarios e a súa información.
- Empregar as sesións para manter a información dos usuarios.
- Xestionar o inicio e peche das sesións dos usuarios.

1.2 Actividade

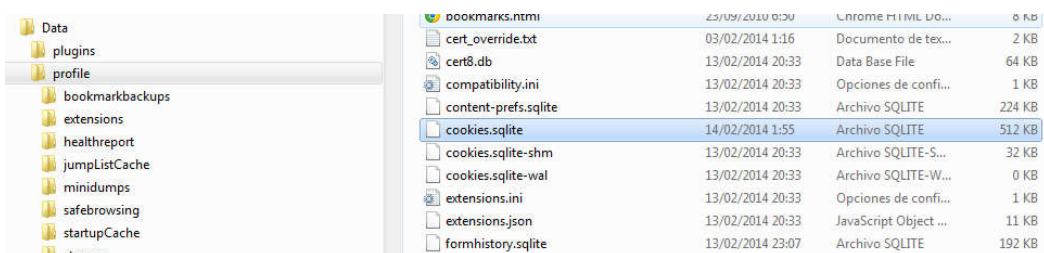
1.2.1 Cookies

Finalidade das *cookies*

Unha *cookie* é unha cantidade limitada de información que garda o navegador no ámbito do usuario. Cada *cookie* está asociada a un sitio web determinado, de tal forma que soamente as súas páxinas podan acceder ao seu contido.

Orixinariamente os navegadores gardaban o contido de cada *cookie* nun pequeno ficheiro de texto. Hoxe en día a maioría de navegadores modernos almacenan a información das *cookies* de xeito centralizado nunha base de datos, pero sempre mantendo independentes unhas de outras, de xeito que cada sitio web poderá acceder soamente as súas *cookies* e non as almacenadas por outros sitios web.

Por exemplo, o navegador Firefox almacena as *cookies* nunha base de datos SQLite dentro do cartafol correspondente ao perfil do usuario.



As especificacións suxiren que os navegadores deben soportar un número mínimo de *cookies* ou unha cantidade mínima de memoria para almacenalas. En concreto, espérase que un navegador sexa capaz de almacenar polo menos 300 *cookies* de 4 Kbytes cada unha e polo menos 20 *cookies* por servidor ou dominio.

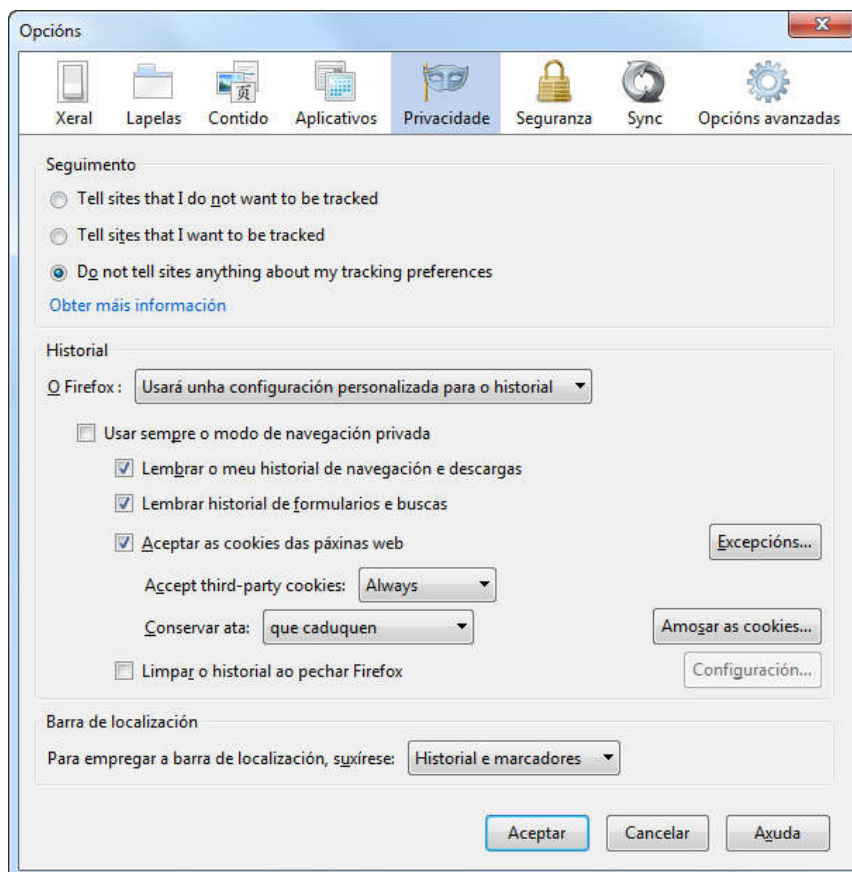
O contido das *cookies* é xestionado polo sitio web que as crea. O seu uso máis típico é o almacenamento das preferencias do usuario (por exemplo, o idioma en que se deben mostrar as páxinas), para que non teña que volver indicalas a próxima vez que visite o sitio.

Cookies e seguridade

Aínda que as *cookies* por si mesmas non son perigosas, algúns sitios web as empregan para rastrexar a navegación dos usuarios. Unha páxina web pode conter imaxes e outros compoñentes que residan noutros dominios; nas conexións que se crean cos servidores destes dominios para obter os compoñentes da páxina tamén se poden crear *cookies*.

Por exemplo, os sitios web que engaden publicidade ás páxinas polas que pasamos poden engadir as súas propias *cookies* e recoller a información daqueles sitios web visitados sempre que tamén conteñan a súa publicidade. A estas *cookies* noméaselles "*cookies* de terceiros" (*third-party cookies*).

Principalmente por este motivo os navegadores inclúen na súa configuración opcións para indicar as *cookies* que queremos almacenar e as que non. Por exemplo, en Firefox podemos atopar esta opción dentro do grupo de "Privacidade".



Hoxe en día atópase vixente a directiva 2009/136/CE do Parlamento Europeo, que obriga aos titulares dos sitios web que empreguen *cookies* a informar adecuadamente aos usuarios e a solicitar o seu consentimento.

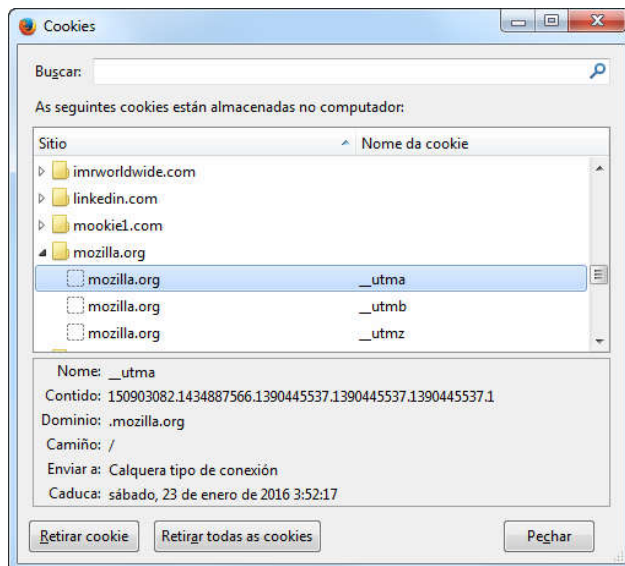
Aproximadamente 16.100 resultados (0,11 segundos)

Las cookies nos permiten ofrecer nuestros servicios. Al utilizar nuestros servicios, aceptas el uso que hacemos de las cookies.

Aceptar

Más información

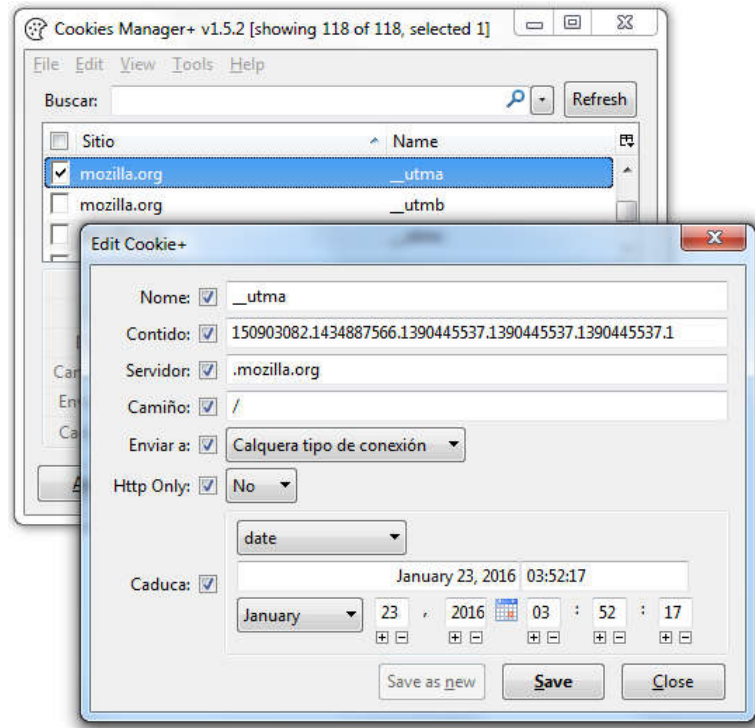
Os navegadores normalmente tamén ofrecen entre as súas opcións a posibilidade de consultar e eliminar as *cookies* que manteñen almacenadas para cada sitio web.



Tamén é habitual atopar extensións que permiten un maior control sobre as *cookies* almacenadas, permitindo editar o seu contido, cambiar o seu dominio, crear copias de seguridade e restauralas, ou incluso crear novas *cookies* dende cero.



Por exemplo, a extensión *Cookies Manager* do navegador Firefox permítenos entre outros ollar e modificar o contido de calquera cookie almacenada no mesmo.

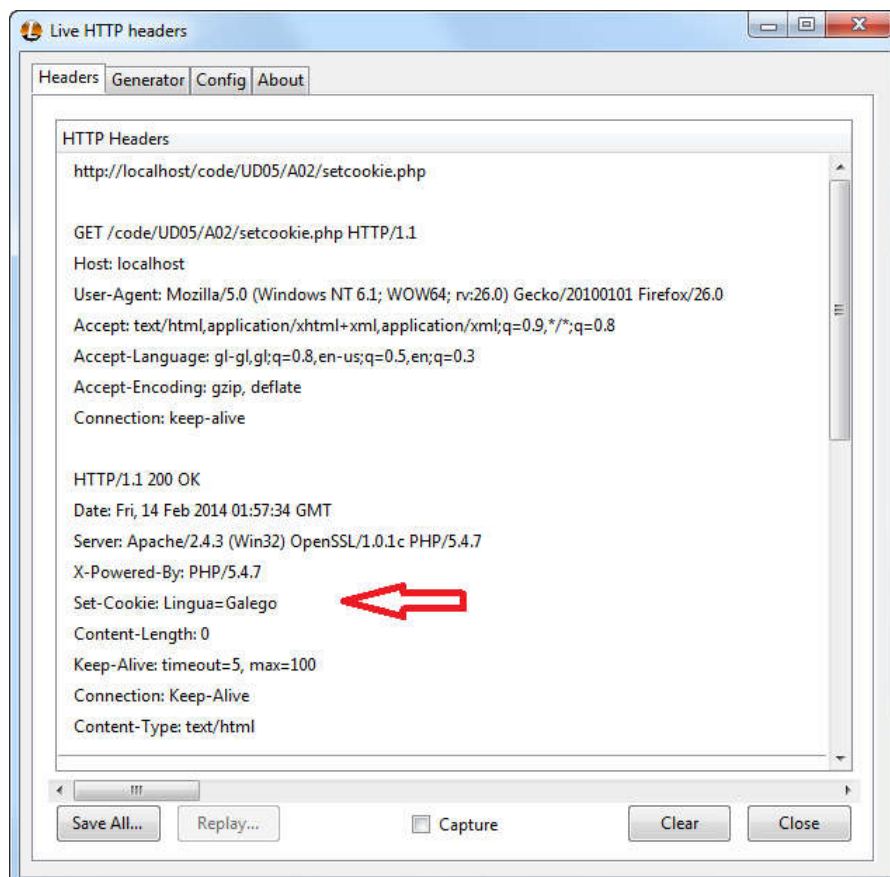


Transmisión das cookies

As *cookies* transmítense empregando encabezados HTTP específicos, tanto nas mensaxes de petición como nas mensaxes de resposta.

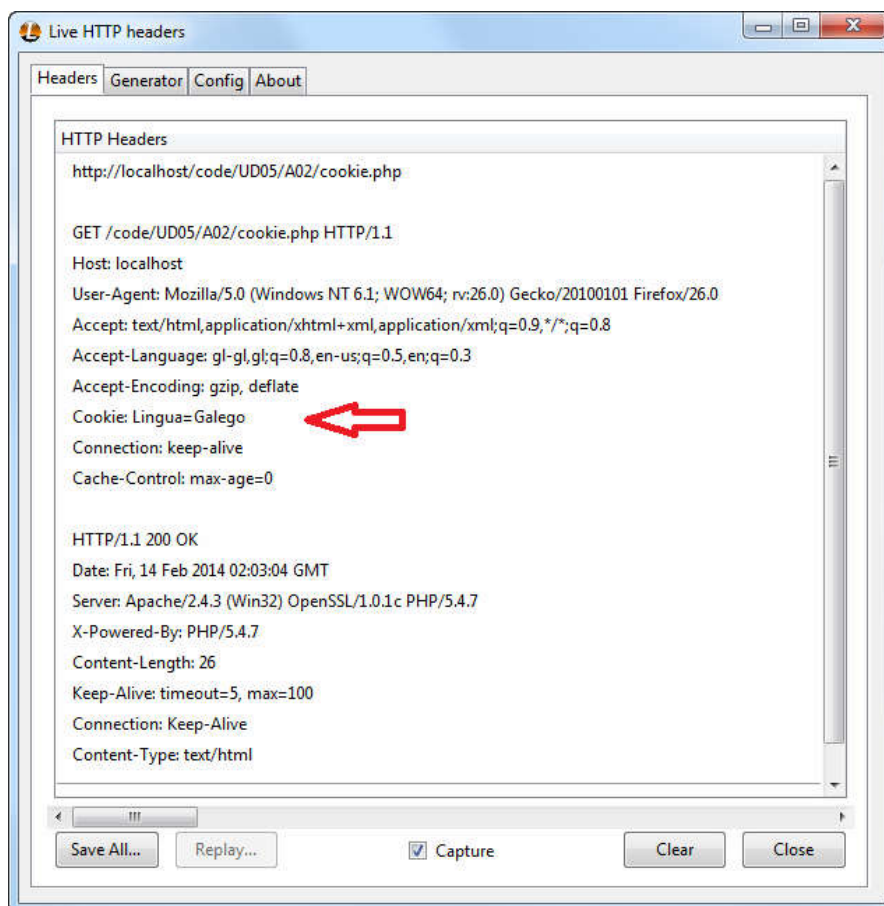
Envío dunha cookie ao navegador

Cando un servidor quere indicar a un navegador que almacene unha *cookie*, envía na mensaxe HTTP de resposta un encabezado "Set-Cookie" cos seus datos

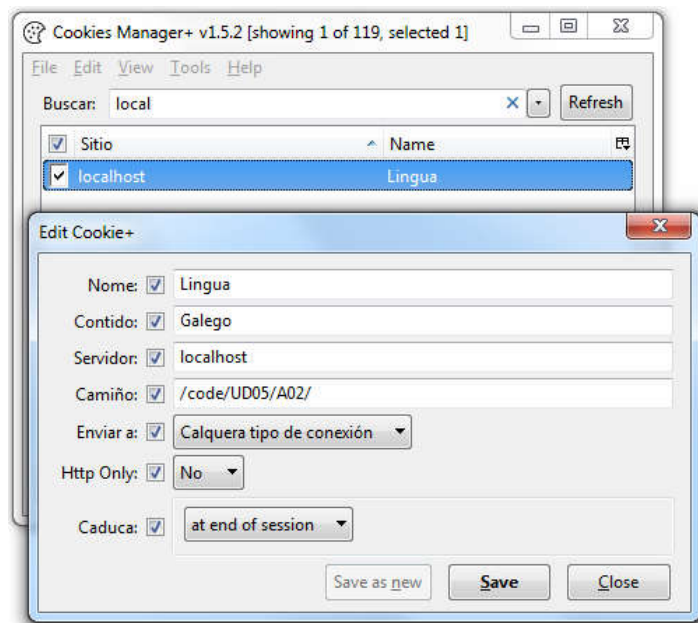


Recuperación das cookies almacenadas no navegador

Cando un navegador envía unha petición solicitando unha páxina dun sitio web, revisa as *cookies* que ten almacenadas e envía ao servidor na mesma mensaxe de petición a información daquelas que corresponda. Faino empregando o encabezado "Cookie" da mensaxe de petición.



Os servidores poden especificar unha data de caducidade para as *cookies*. Se non se especifica data de caducidade, a *cookie* elimínase cando o usuario pecha o navegador (ao "final da sesión"). A aquelas *cookies* que indican unha data de caducidade tamén se lles chama "persistentes", pois non se eliminan ao pechar o navegador. Cando chega a data de vencemento dunha *cookie* persistente, o navegador a elimina de xeito automático.



Non obstante, algúns navegadores como Firefox manteñen as *cookies* daquelas lapelas que estaban abertas cando se pechou o navegador. Polo tanto, é preciso pechar as lapelas antes do navegador para forzar o borrado das *cookies* persistentes.

Emprego de *cookies* en PHP

Almacenamento de información nunha cookie

En PHP, podes utilizar a función "`setcookie`" para almacenar unha *cookie* no navegador do usuario. O único parámetro obrigatorio que tes que usar é o nome da *cookie*, que debe ser unha cadea de texto ASCII que non pode conter os seguintes caracteres: coma ",", punto e coma ";", espazo " " e o signo igual "=". Pero admite varios parámetros máis opcionais: o segundo parámetro é o seu valor e o terceiro é a data de caducidade da *cookie* (é importante subliñar que se trata dunha data, non dun prazo de tempo).

Unha vez almacenada unha *cookie* persistente no navegador é posible eliminar o seu contido antes de que expire. Faise empregando a mesma función "`setcookie`" pero indicando unha data de caducidade anterior á actual).

Por exemplo, se queres almacenar nunha *cookie* a lingua que prefere o usuario para o sitio web, podes facer:

```
setcookie('Lingua', 'Galego');
```

A *cookie* anterior elimínase ao pechar a sesión do navegador. A mesma *cookie* cun prazo de caducidade de unha hora (persistente) faríase:

```
setcookie('Lingua', 'Galego', time()+3600);
```

Empregando o cuarto parámetro da función "`setcookie`" tamén se poden aplicar restricións ás páxinas do sitio que poden acceder a unha *cookie* en función da ruta. Neste caso deberá indicarse como parámetro a ruta dentro do servidor na que a *cookie* estará dispoñible. Se se utiliza a raíz, '/', a *cookie* estará dispoñible na totalidade do sitio. Se se indica unha ruta como '/UD05/', a *cookie* só estará dispoñible dentro do directorio /UD05/ e todos os seus subdirectorios.

```
setcookie('Lingua', 'Galego', time()+3600, '/UD05/');
```

Cando en PHP facemos unha chamada á función "setcookie", o servidor envía un encabezado "Set-Cookie" ao navegador. Por iso, as chamadas á función "setcookie" deben enviarse antes de que o navegador mostre información algunha en pantalla, ao igual que vimos que sucedía coas funcións como "header" que manexaban os encabezados HTTP.

Recuperación da información almacenada nunha cookie

O proceso de recuperación da información que almacena unha *cookie* é moi simple. PHP recolle a información que o servidor recibe nos encabezados "Cookie" das peticións HTTP e a pon accesible aos guións por medio dunha variable superglobal, o array asociativo "\$_COOKIE".

```
echo 'A lingua preferida é ' . $_COOKIE['Lingua'];
```

Sempre que unha aplicación web empregue *cookies*, se debe ter ben presente que en última instancia a súa dispoñibilidade está controlada polo cliente. Por exemplo, algúns usuarios deshabilitan por completo as *cookies* no navegador porque pensan que a información que almacenan pode supoñer un potencial problema de seguridade. Ou a información que almacenan pode chegar a perderse porque o usuario decide formatar o equipo, acceder empregando outro navegador, ou simplemente elimínalas do seu sistema.

Forzar o borrado dunha cookie

Como rematamos de ver, as *cookies* persistentes elimínanse cando vence o seu tempo de vida. Por outra banda, as *cookies* nas que non se especifica o tempo de vida son eliminadas ao pechar a sesión do navegador.

As veces pode acontecer que queiramos obrigar ao borrado dunha *cookie* do navegador. Isto faise reescribindo a mesma *cookie* cun tempo de vida que sinale ao pasado.

Por exemplo, se nalgún momento engadimos a seguinte *cookie*:

```
setcookie('Lingua', 'Galego');
```

E agora queremos eliminala, podemos executar o seguinte código PHP:

```
setcookie('Lingua', 'Galego', time() - 1000);
```

Almacenamento de arrays nas cookies

Aínda que o almacenamento que ofrecen as *cookies* é limitado, pode ser que nalgún intre necesitemos pór nelas os datos dun array. Isto faise engadindo tantas *cookies* como membros teña o array, de unha en unha, indicando os seus nomes coa notación típica dos arrays.

```
<?php
setcookie('usuario[nome]', 'Xaquín');
setcookie('usuario[apelido1]', 'Lorenzo');
setcookie('usuario[apelido2]', 'Fernández');
...
?>
```

Cando máis adiante recuperemos as *cookies*, veremos que se convarten nun array cos seus valores como membros.

```
<?php
if(isset($_COOKIE['usuario'])) {
```

```

foreach ($_COOKIE['usuario'] as $nome => $valor) {
    echo "$nome : $valor <br />";
}
}
?>

```



Na tarefa 1 traballaremos o almacenamento e xestión de información nas *cookies* do navegador.

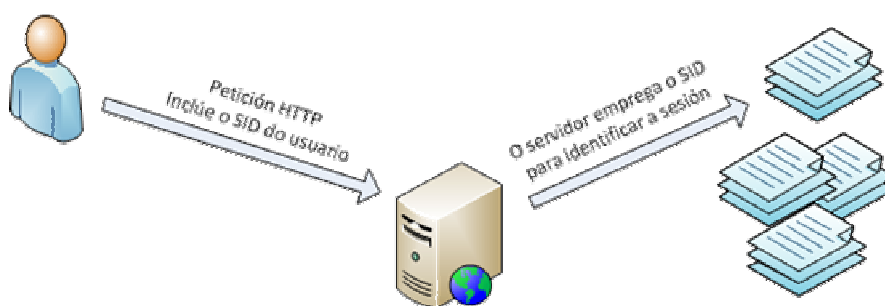
1.2.2 Sesións

Mantemento da información dos usuarios

Como rematamos de ver, unha forma para gardar información particular de cada usuario é utilizar *cookies*. Non obstante, existen diversos problemas asociados ás *cookies*, como o número delas que admite o navegador, ou o seu tamaño máximo.

Para solucionar algúns destes inconvenientes, existen as sesións. No contexto dun servidor web, o termo sesión fai referencia ao conxunto de información relativa a un usuario concreto que este almacena. Esta información pode ser tan simple como o nome do propio usuario, ou máis complexa, como os artigos que depositou na cesta de compra dunha tenda online. Á información que se almacena na sesión dun usuario coñéceselle tamén como *cookies* do lado do servidor (*server side cookies*).

Cada usuario distinto dun sitio web ten a súa propia información de sesión. Para distinguir unha sesión doutra úsanse os identificadores de sesión (SID, *Session ID*). Un SID é un atributo único que se asigna a cada un dos visitantes dun sitio web (cando o visitan por primeira vez ou a primeira vez que se almacenan datos na súa sesión) e identifícao. Desta forma, se o servidor web coñece o SID dun usuario, pode relacionalo con toda a información que posúe sobre el, a que se mantén na sesión do usuario. Esa información almacénase no servidor web, xeralmente en ficheiros aínda que tamén se poden utilizar outros mecanismos de almacenamento como bases de datos.



Polo tanto, o SID dun usuario é o atributo que o identifica e que emprega o servidor web para recuperar a súa información almacenada na sesión. E dado que o protocolo HTTP é un protocolo sen estado (cada petición é independente das anteriores), o usuario terá que identificarse enviando o seu SID ao servidor en cada petición que faga.

Isto é, o navegador do usuario terá que:

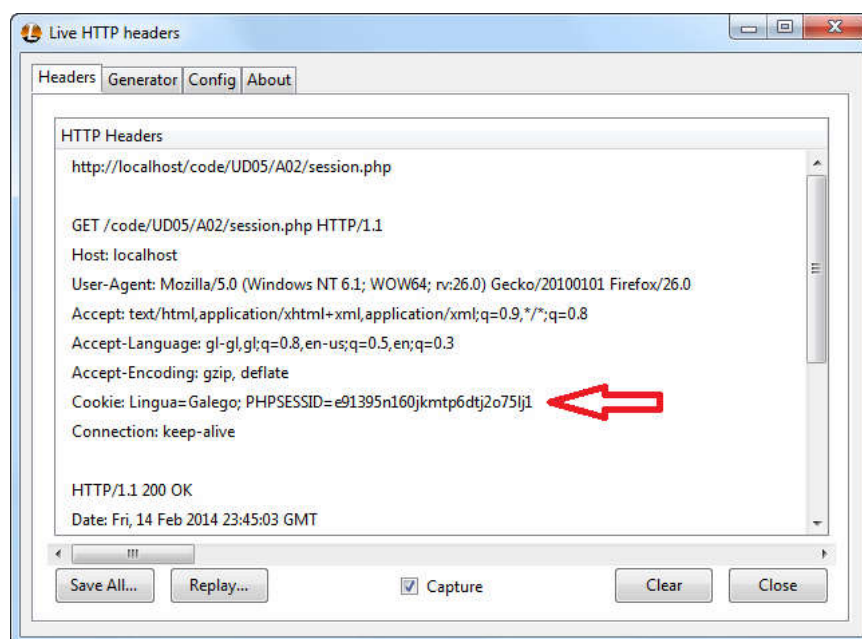
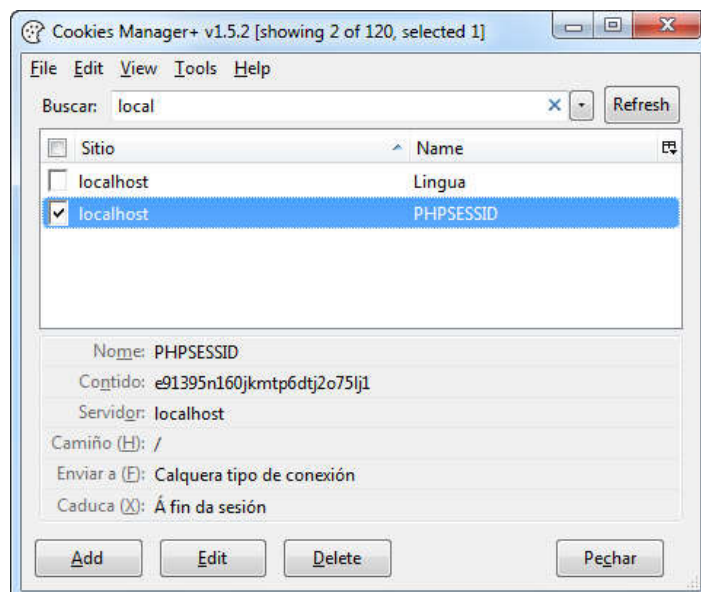
- Coñecer o SID do usuario.
- Enviar o SID ao servidor web xunto con cada novo mensaxe petición.

E cando o servidor web reciba o SID do usuario:

- Recuperará a súa información de sesión, que poderá modificar ou empregar para xerar o contido da mensaxe de resposta.

E onde se atopa ese SID, o identificador da sesión, que é único para cada usuario? Pois existen dúas maneiras de manter o SID entre as páxinas dun sitio web que visita o usuario:

- Utilizando *cookies*. O servidor web crea unha *cookie* no navegador para almacenar o identificador do usuario. En PHP normalmente o nome desa *cookie* é "PHPSESSID".



- Propagando o SID nun parámetro da URL, que en PHP tamén adoita ser "PHPSESSID". O SID engádese como unha parte máis da URL, da forma:

`http://www.misitioweb.com/tienda/listado.php&PHPSESSID=34534fg4ffg34ty`



Ningunha das dúas maneiras é perfecta. Xa sabes os problemas que ten a utilización de *cookies*. Malia iso, é o mellor método e o máis utilizado. Propagar o SID como parte da URL leva consigo maiores desvantaxes, como que sexa visible na barra de direccións do navegador ou o feito de que compartir a URL con outra persoa implica compartir tamén o identificador de sesión.

É importante ter en conta que aínda que a información da sesión non viaxa entre o cliente e o servidor, si o fai o SID, ben como parte da URL ou nun encabezado HTTP no caso de que se garde nunha *cookie*. En ambos os dous casos, isto formula un posible problema de seguridade. O SID pode ser conseguido por outra persoa, e a partir deste obter a información da sesión do usuario. O xeito máis seguro de utilizar sesións é almacenando os SID en *cookies* e utilizar HTTPS para encriptar a información co SID que se transmite entre o servidor web e o cliente.

A boa noticia, é que o proceso de manexo de sesións en PHP está automatizado en boa medida. Cando un usuario visita un sitio web, non é necesario programar un procedemento para ver se existe un SID previo e cargar os datos asociados con este. Tampouco é preciso utilizar a función "setcookie" se queres almacenar os SID en *cookies*, ou ir pasando o SID entre as páxinas web do teu sitio se te decides por propagalo. Todo isto PHP faino automaticamente.

Configuración das sesións

Antes de utilizar sesións nun sitio web, é importante configurar correctamente PHP utilizando as seguintes directivas no ficheiro "php.ini" segundo corresponda:

Directivas de sesións en PHP	
session.use_cookies	Indica se se deben usar cookies (1) ou propagación na URL (0) para almacenar o SID.
session.use_only_cookies	Débese activar (1) cando se empregan cookies para almacenar os SID, para indicar que non se recoñezan os SID que se poidan pasar como parte da URL (estes SID pódense usar para usurpar o identificador doutro usuario).
session.save_handler	Utilízase para indicar a PHP como debe almacenar os datos da sesión do usuario. Existen catro opcións: en ficheiros (files), en memoria (mm), nunha base de datos SQLite (sqlite) ou utilizando para iso funcións que debe definir o programador (user). O valor por defecto (files) funcionará sen problemas na maioría dos casos.
session.save_path	A ruta na que se gardarán as sesións dos usuarios, no caso de empregar almacenamento en ficheiros.
session.name	Determina o nome da cookie que se utilizará para gardar o SID. O seu valor por defecto é "PHPSESSID".
session.auto_start	O seu valor por defecto é 0, e neste caso será necesario usar a función session_start para xestionar o inicio das sesións. Cando un sitio web emprega sesións con frecuencia, pode ser boa idea cambiar o seu valor a 1 para que PHP active de forma automática o manexo de sesións.
session.cookie_lifetime	Establece o tempo en segundos que se manterá a cookie da sesión no navegador do usuario, sempre que se empregue ese método de transmisión do SID. O seu valor por defecto é 0, co cal a cookie manterase ata que se peche o navegador. Cando se emprega a URL para propagar o SID, este perderase cada vez que o usuario peche o navegador.
session.cookie_secure	Cando se activa obriga a que as cookies de sesión soamente se transmitan a través de conexións seguras HTTPS. Deste xeito asegúrase o SID fronte a posibles ataques de interceptación.
session.gc_maxlifetime	Indica o tempo en segundos que se debe manter a información da sesión, aínda que non haxa ningunha actividade por parte do usuario. O seu valor por defecto é 1440. É dicir, pasados 24 minutos dende a última actividade por parte do usuario, os datos da súa sesión pasan ao recolector de lixo (<i>garbage collector</i> , gc).
session.use_trans_sid	Activa ou desactiva o soporte de PHP para o envío transparente do SID na URL. Cando se activa, PHP engade de xeito automático o parámetro PHPSESSID aos enlaces a rutas relativas.
url_rewriter.tags	Cando a directiva anterior atópase activada, esta indica a lista de etiquetas nas que se lles engadirá de xeito transparente o parámetro co SID nas URLs.

As directivas da táboa anterior poden establecerse de calquera das seguintes formas:

- No arquivo global de configuración de PHP "php.ini".
- Dentro dos arquivos "httpd.conf" e ".htaccess", no caso de empregar Apache como servidor web.
- Nun arquivo de configuración ".user.ini", semellante aos ".htaccess" de Apache.
- Dentro dun guión PHP, empregando unha función "ini_set" ou semellante.

A excepción é a directiva "session.auto_start", que non pode establecerse desde dentro dun guión.

Xestión da información das sesións

O inicio dunha sesión pode ter lugar de dúas formas:

- Se a directiva "session.auto_start" está activa, a sesión comezará automaticamente en canto un usuario se conecte ao sitio web.

En caso de que ese usuario xa abraise unha sesión con anterioridade, e esta non se eliminase, en lugar de abrir unha nova sesión continuarase a anterior. Para iso utilízase o SID anterior, que deberá estar almacenado nunha *cookie* pois cando se emprega propagación do SID non é posible restaurar sesións anteriores (o SID figura na URL e pérdese cando se pecha o navegador).

- Se pola contra, non se atopa activo o inicio automático de sesións, será necesario executar a función "session_start" para indicar a PHP que inicie unha nova sesión ou continúe a anterior. Aínda que anteriormente esta función devolvía sempre "true", a partir da versión 5.3.0 de PHP o seu comportamento é máis coherente: devolve "false" en caso de non poder iniciar ou restaurar a sesión.

Como normalmente o inicio de sesión require utilizar *cookies*, e estas transmítense nos encabezados HTTP, deberase ter en conta que para poder iniciar unha sesión mediante "session_start" haberá que facer as chamadas a esta función antes de que a páxina web mostre información no navegador (do mesmo xeito que se facía coas chamadas a "setcookie". Ademais, todas as páxinas que necesiten utilizar a información almacenada na sesión, deberán executar a función "session_start".

Mentres a sesión permanece aberta, podes utilizar a variable superglobal "\$_SESSION" para engadir información á sesión do usuario, ou para acceder á información almacenada na sesión. Por exemplo, o seguinte guión conta o número de veces que o usuario visita a páxina, gardando o contador na sesión do usuario:

```
<?php
// Iniciamos a sesión ou recuperamos a anterior sesión existente
session_start();

// Comprobamos se a variable xa existe
if (isset($_SESSION['visitas'])) $_SESSION['visitas']++;
else $_SESSION['visitas'] = 0;

?>
```

Se en lugar do número de visitas, se quixera almacenar o instante en que se produce cada unha, a variable de sesión "visitas" deberá ser un array e polo tanto habería que cambiar o código anterior por:

```
<?php
// Iniciamos a sesión ou recuperamos a anterior sesión existente
```

```

session_start();

// En cada visita engadimos un valor ao array "visitas"
$_SESSION['visitas'][] = mktime();

?>

```

Tamén é posible almacenar obxectos na sesión do usuario. Faise como con calquera outra variable:

```

<?php
// Iniciamos a sesión ou recuperamos a anterior sesión existente
require_once('clases/Visita.php');
session_start();
// Creamos un novo obxecto e o gardamos na sesión
$o = new Visita();
$o->setTempo(mktime());
$_SESSION['visita'] = $o;

?>

```

Cando almacenamos un obxecto na sesión, PHP o serializa de xeito automático. E do mesmo xeito, cando recuperamos un obxecto da sesión do usuario, PHP o deserializa por nós.

```

<?php
// Iniciamos a sesión ou recuperamos a anterior sesión existente
require_once('clases/Visita.php');
session_start();
// Recuperamos o obxecto da sesión
$o = $_SESSION['visita'];

?>

```

É importante cargar a definición da clase antes de recuperar o obxecto serializado na sesión (de feito, antes de iniciar a sesión), pois esta non se garda na variable superglobal "\$_SESSION".

Peche manual da sesión

Aínda que xa vimos que é posible configurar PHP para que elimine de forma automática os datos dunha sesión pasado certo tempo, en ocasións pode ser necesario pechala de forma manual nun momento determinado. Por exemplo, cando se empregan sesións para recordar a información de autenticación, convén darlle ao usuario do sitio web a posibilidade de pechar a sesión cando o crea conveniente.

En PHP poden empregarse dúas funcións para eliminar a información almacenada na sesión:

- `session_unset`. Elimina as variables almacenadas na sesión actual, pero non elimina a información da sesión do dispositivo de almacenamento usado.
- `session_destroy`. Elimina completamente a información da sesión do dispositivo de almacenamento.

Almacenamento da información da sesión

Nalgúns casos pode ser convinte gardar nun ficheiro ou nunha base de datos a información que contén a sesión do usuario, para poder recuperala máis adiante. Deste xeito, por exemplo, un usuario que pecha a súa sesión nunha tenda web, cando regrese pode ter a cesta da compra tal e como a deixou.

En PHP temos dúas funcións que nos facilitan o procedemento:

- `session_encode`. Devolve a información da sesión codificada nunha cadea de texto.
- `session_decode`. Recibe unha cadea de texto coa información da sesión, e co seu contido rexenera o array `$_SESSION`.



Na tarefa 2 crearemos guións PHP para traballar co almacenamento de información na sesión do usuario.

Secuestro e fixación da sesión

Existen dous tipos distintos de ataques que se basean no identificador de sesión: o secuestro e a fixación da sesión.

Secuestro da sesión

Chamamos secuestro da sesión a calquera método ou técnica que teña como fin conseguir o identificador da sesión dun usuario. As formas de conseguir este obxectivo son variadas, como por exemplo interceptando o tráfico da rede, ou incluso realizando un ataque de forza bruta, probando moitos SIDs ata conseguir un válido.

Para evitar na medida do posible este tipo de ataques, podemos empregar as seguintes medidas:

- Unha das primeiras medidas para evitar o secuestro da sesión dos usuarios é enviar os identificadores do cliente ao servidor empregando sempre *cookies*. Deste xeito, o identificador non aparece na URL de cada unha das páxinas, o que podería ocasionar o envío ou a publicación accidental do mesmo polo usuario ao querer obter o enlace dunha páxina calquera. Isto pode acadarse empregando a directiva de configuración "`session.use_only_cookies`", que a partires da versión 5.3.0 xa se atopa por defecto a '1'.

```
session.use_only_cookies = 1
```

- Cando sexa posible tamén é recomendable empregar conexións seguras con HTTPS para evitar a interceptación do tráfico.
- Outra medida para dificultar o secuestro da sesión é limitar o seu tempo de vida. Isto pode facerse de dúas maneiras:
 - Empregando a directiva "`session.cookie_lifetime`" para indicar o tempo de caducidade dunha sesión. Transcorrido ese tempo, a sesión pasa a considerarse inválida e deberá abrirse unha nova. Isto mesmo pode facerse coa función "`session_set_cookie_params`".

```
<?php
session_set_cookie_params('600'); // 10 minutos
session_start();
...
```

?>

- Empregando a directiva "session.gc_maxlifetime", que indica o tempo de inactividade dunha sesión. Pasado ese tempo, a información da sesión pasa a considerarse inválida.
- Tamén podemos cambiar o ID da sesión de cando en vez empregando a función "session_regenerate_id". Este método é transparente para os usuarios activos pero provoca a invalidez do anterior identificador, que xa non poderá empregarse con fins maliciosos en caso de ter sido capturado.

```
<?php
session_start();
session_regenerate_id();
...
?>
```

- Para rematar, outra medida é comprobar en cada petición os datos relacionados co cliente, como poden ser o axente de usuario (\$_SERVER['HTTP_USER_AGENT']) e en casos extremos tamén a dirección orixe da petición (\$_SERVER['REMOTE_ADDR']). Cando algunha das dúas cambia con respecto á obtida na anterior petición, sobre todo a primeira, debería anularse a sesión e forzar ao usuario a rexistrarse de novo para comezar unha nova sesión.

```
<?php
session_start();
if ($_SERVER['HTTP_USER_AGENT'] != $_SESSION['USER_AGENT_ANTERIOR']) {
    session_destroy(); // Eliminamos a información da sesión
}
session_regenerate_id(); // Xeramos un novo SID
$_SESSION['USER_AGENT_ANTERIOR'] = $_SERVER['HTTP_USER_AGENT'];
...
?>
```

Fixación da sesión

No método de secuestro da sesión se descoñece o identificador do cliente e o que se busca é tratar de descubrir cal é. Nesta ocasión, o que se busca é acadar que o usuario se loguee cun identificador de sesión que xa coñecemos.

A forma máis habitual deste ataque é facer que o usuario empregue unha URL que conteña un identificador de sesión como a seguinte.

```
http://www.misitioweb.com/tienda/listado.php&PHPSESSID=34534fg4ffg34ty
```

Se o sitio web acepta URLs como a anterior, co SID incorporado, cando o usuario pinche no enlace abrirá unha nova sesión co identificador que figura na URL. Se a continuación o usuario inicia sesión no sitio web, o atacante xa dispón dun SID correspondente a unha sesión rexistrada.

Para evitar estes ataques deben terse en conta as mesmas recomendacións mencionadas anteriormente para evitar o secuestro da sesión, e tamén revisar o valor do parámetro "session.use_trans_sid" na configuración de PHP.

```
session.use_trans_sid = 0;
```

Se o valor do parámetro anterior fose '1', PHP lería os identificadores de sesión pasados nun parámetro GET da URL, aínda que o usuario estivese a empregar *cookies* para a súa transmisión. Isto posibilitaría a creación de URLs fraudulentas como no caso anterior.

1.2.3 Autenticación de usuarios

As contrasinais

Moitas aplicacións web, estean ou non programadas en PHP, empregan dun ou doutro xeito contas de usuario que, maiormente, adoitan usar contrasinais para rexistrarse e autenticarse.

E como almacenan estas aplicacións web os contrasinais para autenticar aos usuarios? Pois a resposta é que non o fan. O contrasinal é unha información moi sensible e como tal non debe ser almacenada. En vez diso, o que se soe facer é almacenar os "*hashes*" dos contrasinais. Sen *hashing*, calquera contrasinal que se almacene na aplicación poderá ser roubado se a base de datos se ve comprometida, co que inmediatamente non só estaría comprometida a aplicación, senón tamén as contas doutros servizos dos nosos usuarios, sempre e cando non utilicen contrasinais distintos. Se aplicamos un algoritmo *hash* aos contrasinais antes de almacenalos, dificultamos ao atacante determinar o contrasinal orixinal, malia que nun futuro poderá comparar o *hash* resultante co contrasinal orixinal.

Isto é, cando se rexistre un novo usuario na aplicación web deberemos almacenar o resultado de pasar o contrasinal do usuario por unha función *hash*, e posteriormente empregar a mesma función para validar o contrasinal introducido polo usuario cada vez que este inicie unha nova sesión.

O salt

Existen moitas funcións que nos permiten obter un *hash* dun contrasinal. Con todas elas, é recomendable empregar un "*salt*" xunto co contrasinal do usuario para obter o hash. Un *salt* criptográfico é un dato que se utiliza durante o proceso de *hash* para eliminar a posibilidade de que o resultado poida buscarse a partir dunha lista de pares precalculados de *hash* e as súas entradas orixinais, coñecidas como táboas "*rainbow*".

É dicir, un *salt* é un pequeno dato engadido que fai que os *hashes* sexan moito máis difíciles de *crackear*. Existe un gran número de servizos online que ofrecen grandes listas de códigos *hash* precalculados, xunto cos seus datos de entrada orixinais. O uso dun *salt* fai moi difícil ou imposible encontrar o *hash* resultante en calquera destas listas.

O *salt* que empreguemos condicionará os resultados *hash* obtidos. Por isto é importante empregar o mesmo *salt* tanto no rexistro dun usuario como na comprobación do seu contrasinal nos sucesivos accesos á aplicación web.



No TA1 veremos as opcións que temos dentro da librería de funcións de PHP para obter unha cadea *hash* a partires dun texto, nomeadamente os contrasinais dos usuarios.

Mantemento e peche da sesión do usuario

Como xa sabemos, os datos da sesión do usuario gárdanse no servidor, e o SID que identifica cada unha das sesións almacénase normalmente nunha *cookie* do navegador do usuario.

Tamén vimos que esta *cookie* ten un tempo de vida definido por un parámetro da configuración de PHP, `session.cookie_lifetime`, que en principio é "0" (mantense ata que se peche o navegador do usuario) pero podemos axustalo para facer que o navegador borre

a *cookie* pasado un certo tempo, e polo tanto invalide a sesión do usuario. E ademais temos outro parámetro de configuración, `session.gc_maxlifetime`, que fixa un prazo de validez para a información da sesión almacenada no servidor web.

Polo tanto podemos supoñer que empregando calquera deses dous parámetros, poderíamos definir un prazo de tempo pasado o cal a información sesión do usuario pasaría a ser invalidar ou a estar eliminada. Pero non é tan doado:

- Aínda que definamos un prazo de validez para a *cookie* da sesión do usuario, ese prazo depende da xestión das *cookies* que faga o navegador do usuario, e xa vimos que existen métodos para alongar ese prazo ou invalidalo.
- Pasado o tempo indicado en `session.gc_maxlifetime`, a información da sesión é considerada inválida e pasa a mans do GC (*Garbage Collector*), pero iso non significa que sexa borrada de inmediato. De feito, este soamente se inicia ou non dependendo dunha probabilidade fixada por outros parámetros da configuración.

Polo tanto, imos ver dous métodos para:

- Eliminar de xeito automático a información da sesión do usuario pasado un certo tempo sen actividade pola súa parte.
- Manter aberta a sesión do usuario aínda que este peche o seu navegador.

Combinando estes dous métodos podemos programar aplicacións web con sesións persistentes, que o usuario poda pechar de xeito manual ou que se pechen soas pasado un certo tempo, pero que se poidan manter abertas aínda cando este peche e abra de novo o navegador.

Establecer un tempo límite de inactividade

O mellor neste caso é xestionar o tempo de inactividade gardando nunha variable da sesión do usuario o instante no que se produciu a última acción. A seguinte vez que o usuario intente recuperar unha páxina do servidor, haberá que calcular o tempo transcorrido para ver se procede ou non pechar a sesión e eliminar a súa información.

```
<?php
    session_start();

    $max_seg_inactivo = 600;

    if (isset($_SESSION['ultima_actividade']) &&
        (time() - $_SESSION['ultima_actividade'] > $max_seg_inactivo)) {
        session_unset();
        session_destroy();
    }

    $_SESSION['ultima_actividade'] = time();
    ...
?>
```

Manter aberta a sesión aínda que se peche o navegador do usuario

Os identificadores das sesións dos usuarios almacénanse maiormente en *cookies* dos navegadores, e estas teñen por defecto un tempo de vida "0", o que quere dicir que estas *cookies* elimínanse cando se pecha o navegador. Polo tanto, cada vez que o usuario pecha o navegador, pérdense os seus identificadores de sesión e tamén calquera información asociada que se atope almacenada no servidor. E cando ese mesmo usuario abra de novo o navegador, atoparase cunha sesión baleira (por exemplo, deberá identificarse de novo se é o caso).

Nalgúns casos pode ser convinte preparar a nosa aplicación web para que non se perda a información da sesión do usuario cada vez que se pecha o seu navegador. Deste xeito, por exemplo, calquera usuario que se atope rexistrado na nosa aplicación, poderá seguir rexistrado tras reiniciar o navegador.

Para evitar que a información da sesión do usuario se elimine cando este peche o seu navegador, unha posibilidade é a que xa falamos: cambiar o parámetro `session.cookie_lifetime` da configuración de PHP para acadar que a *cookie* co SID do usuario teña un tempo de vida distinto de "0", e polo tanto non se destrúa ao pechar o navegador. Pero o principal problema deste cambio é que afecta a tódalas aplicacións que se executen no mesmo servidor, e en moitas ocasións (aloxamento compartido) non teremos permisos para facelo.

Polo tanto, unha opción mellor é gardar pola nosa conta no navegador certa información correspondente ao usuario que nos permita restaurar a sesión despois de pechada. Isto é, deberemos empregar *cookies* propias da aplicación web con algunha información que nos permita identificalo de novo cando se volva a conectar. Por exemplo, estas *cookies* poderán conter:

- O nome ou id do usuario.

```
setcookie("login", $nomeusuario, time()+3600*24*14);
```

- Un *hash* calculado a partires cando menos do nome/id do usuario e un *salt*. É necesario para evitar que o usuario obteña a sesión doutro usuario simplemente cambiando o nome gardado na *cookie* anterior.

```
$salt = '...';  
setcookie("hash", crypt($nomeusuario, $salt), time()+3600*24*14);
```

Deste xeito, cando o usuario volva a abrir o navegador e conectarse á aplicación web, haberá que:

- Coller da *cookie* o seu nome de usuario.

```
$nomeusuario = $_COOKIE['login'];
```

- Coller o *hash* almacenado tamén na *cookie*, e ver se coincide cos nosos cálculos. Se é así, podemos proceder a iniciarlle sesión de xeito automático (e opcionalmente recuperar calquera outra información da sesión que teñamos gardada, como a lista de artigos no seu carro da compra).

```
$hash = $_COOKIE['hash'];  
if($hash == crypt($nomeusuario, $salt)) {  
    // Iniciamos sesión para o usuario  
    $_SESSION['hash'] = $nomeusuario;  
    ...  
}
```

Podemos mellorar o procedemento engadindo ao *hash* outros parámetros como:

- A dirección IP do usuario.
- O axente de usuario do navegador do usuario.

```
$salt = '...';  
$texto = $nomeusuario.$_SERVER['HTTP_USER_AGENT'].$_SERVER['REMOTE_ADDR'];  
setcookie("hash", crypt($texto, $salt), time()+3600*24*14);
```

Deste xeito, se cambia algún dos parámetro anteriores o *hash* almacenado non coincidirá co que calculemos nós e o usuario terá que introducir de novo as súas credenciais para iniciar sesión.



Na tarefa 3 crearemos guións PHP que empreguen a información das sesións para autenticar aos seus usuarios.

1.3 Tarefas

As tarefas propostas son as seguintes.

- Tarefa 1. **Almacenamento de información nas *cookies***. Nesta tarefa traballaremos coa xestión das cookies presentes no navegador do usuario e o manexo da súa información.
- Tarefa 2. **Almacenamento de información na sesión do usuario**. Nesta tarefa xestionaremos a información presente na sesión do usuario.
- Tarefa 3. **Autenticación de usuarios**. Nesta tarefa crearemos unha aplicación web sinxela que implemente o rexistro e a autenticación dos seus usuarios.

1.3.1 Tarefa 1. Almacenamento de información nas *cookies*

a) Tarefa 1_a

Imos comezar creando unha pequena aplicación web que:

- Amose as *cookies* recibidas do navegador.
- Incorpore un formulario que permita introducir unha nova *cookie* no navegador, indicando o seu nome e o seu valor.

As novas *cookies* introducidas no formulario deberán amosarse a continuación na lista da mesma *páxina*, do seguinte xeito.



b) Tarefa 1_b

Agora imos engadirlle un par de detalles á páxina anterior:

- Deberán poderse introducir *cookies* sen valor, soamente co seu nome.
- No formulario haberá un dato máis, o tempo de vida da *cookie* que se indicará en segundos. Cando se cubra ese campo, a *cookie* que se envíe deberá ser persistente.

c) Tarefa 1_c

Imos continuar coa mesma páxina. Nesta ocasión temos que crear un novo botón que permita eliminar todas as *cookies* do navegador do usuario. Por exemplo:



Ao premer o botón de borrado, recargarase a páxina e a lista de *cookies* deberá quedar baleira.

Autoavaliación

a) Tarefa 1_a

Podemos programar a aplicación nunha única páxina cun código semellante ao seguinte:

```
<?php
    if(!empty($_POST['nome']) && !empty($_POST['valor'])) {
        setcookie($_POST['nome'], $_POST['valor']);
        header('Location: cookies.php');
    }
?>
<!DOCTYPE html>
<html>
    <head>
        <title>Información nas cookies</title>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <style type="text/css">
            #novacookie {
                width: 200px;
                float: left;
                margin-left: 20px;
            }
            input[type=text] {
                width: 80%;
```

```

        display: block;
    }
</style>
</head>
<body>
    <h1>Información nas cookies:</h1>
    <?php var_dump($_COOKIE); ?>
    <div id="novacookie">
        <form method='post'>
            <p>Nova cookie:</p>
            <fieldset>
                <label for="nome">Nome:</label>
                <input id="nome" type="text" name="nome" />
                <label for="valor">Valor:</label>
                <input id="valor" type="text" name="valor" />
            </fieldset>
            <fieldset>
                <input type="submit" value="Enviar" />
            </fieldset>
        </form>
    </div>
</body>
</html>

```

O formulario envía o nome e o valor da nova *cookie* que temos que almacenar no navegador á mesma páxina, que cando atopa os datos fai unha chamada á función `setcookie`. Isto fai que a nova *cookie* se envíe ao navegador cos encabezados da páxina. Pero o array superglobal `$_COOKIE` aínda non recibiu do navegador a nova *cookie*, e polo tanto non a vai a amosar.

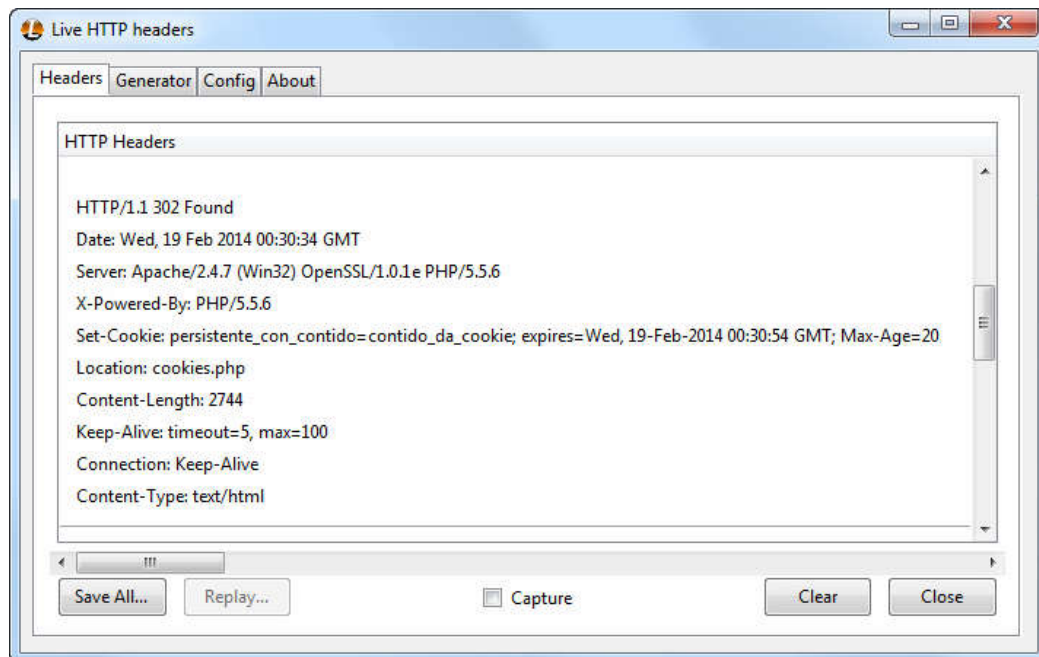
A solución é simplemente refrescar a páxina empregando un encabezado HTTP `header`. Deste xeito na seguinte petición que se recibe dende o navegador xa figura a nova *cookie* e se amosará xunto coas outras.

Pódese comprobar este comportamento eliminando a liña que envía o encabezado `header` ao navegador.

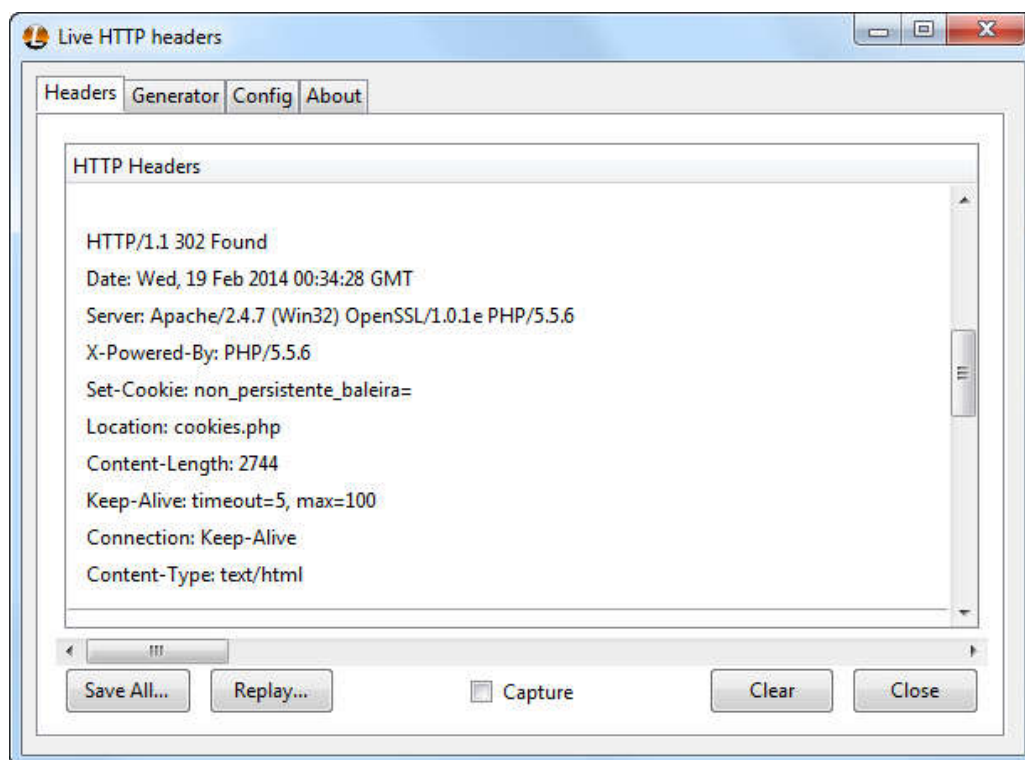
b) Tarefa 1_b

Aínda que é normal crear unha *cookie* baleira, que soamente conteña o nome, no caso das *cookies* persistentes (aquelas que especifican unha data límite de validez) isto non é posible.

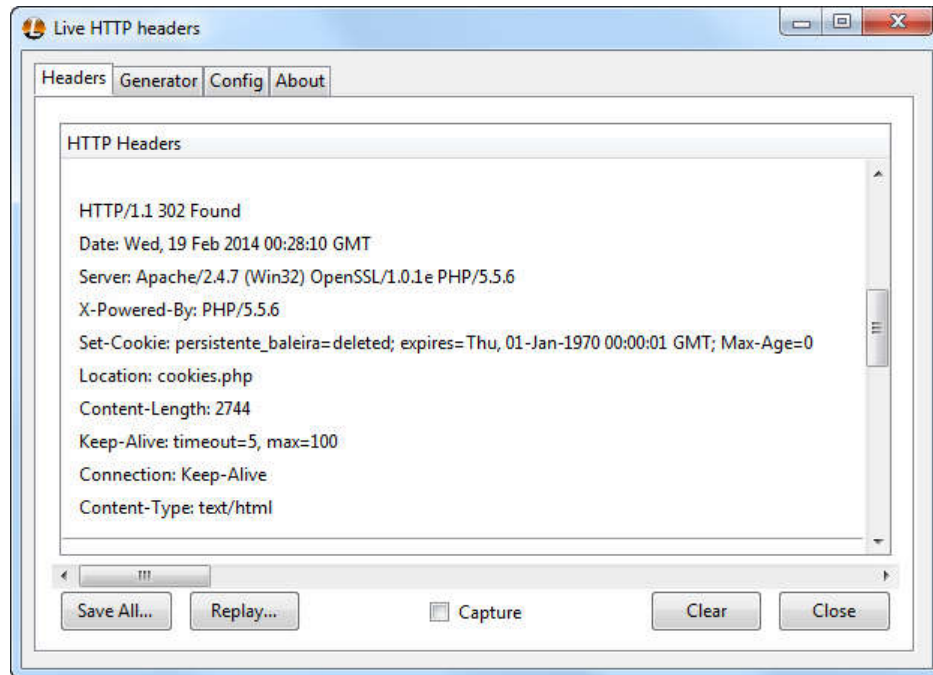
Cando creamos unha *cookie* persistente non baleira, o servidor a envía de xeito correcto ao navegador.



E o mesmo acontece cando enviamos unha *cookie* baleira non persistente.



Pero cando intentamos facer o mesmo cunha *cookie* persistente baleira, o servidor envía realmente un aviso ao navegador para que elimine a *cookie*.



Isto é, cando unha cookie non ten valor, non poderá ser persistente. Polo tanto, podemos solucionar a tarefa modificando o código anterior da seguinte forma.

```
<?php
if(!empty($_POST['nome'])) {
    if(empty($_POST['valor'])) {
        if(empty($_POST['segs'])) setcookie($_POST['nome']);
        else {
            // Non podemos crear cookies baleiras persistentes
            // polo que creamos a cookie cun valor '0'
            setcookie($_POST['nome'], '0', time()+$_POST['segs']);
        }
    }
}
else {
    if(empty($_POST['segs'])) setcookie($_POST['nome'], $_POST['valor']);
    else setcookie($_POST['nome'], $_POST['valor'], time()+$_POST['segs']);
}

header('Location: cookies.php');
}
?>

<!DOCTYPE html>
<html>
    <head>
        <title>Información nas cookies</title>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <style type="text/css">
            #novacookie {
                width: 200px;
                float: left;
                margin-left: 20px;
```

```

    }
    input[type=text] {
        width: 80%;
        display: block;
    }
</style>
</head>
<body>
    <h1>Información nas cookies:</h1>
    <?php var_dump($_COOKIE); ?>
    <div id="novacookie">
        <form method='post'>
            <p>Nova cookie:</p>
            <fieldset>
                <label for="nome">Nome:</label>
                <input id="nome" type="text" name="nome" />
                <label for="valor">Valor:</label>
                <input id="valor" type="text" name="valor" />
                <label for="segs">Segundos de permanencia:</label>
                <input id="segs" type="text" name="segs" />
            </fieldset>
            <fieldset>
                <input type="submit" value="Enviar" />
            </fieldset>
        </form>
    </div>
</body>
</html>

```

Pódese comprobar que cando creamos *cookies* cun tempo de vida definido, éstas desaparecen ao refrescar a pantalla do navegador pasado ese tempo.

c) Tarefa 1_c

Podemos acadar o obxectivo engadindo un pequeno formulario a continuación do que xa tiñamos.

```

<div id="borracookies">
    <form method='post'>
        <fieldset>
            <input type="hidden" name="borracookies" value="borra" />
            <input type="submit" value="Borrar as cookies" />
        </fieldset>
    </form>
</div>

```

Podemos engadir tamén algún estilo para que posicione o novo "<div>" ao carón do anterior.

```

<style type="text/css">
...
#borracookies {
    width: 100px;
    float: left;
    margin: 60px 0 0 40px;
}
</style>

```

E ao comezo da páxina teremos que engadir o seguinte código PHP para comprobar se foi pulsado o botón de borrado, e nese caso procesar a lista de cookies poñéndolles un tempo de vida no pasado.

```

<?php
...

if(!empty($_POST['borracookies'])) {
    foreach($_COOKIE as $cookie => $valor) {
        setcookie($cookie, '', time()-1000);
    }
    header('Location: cookies.php');
}
?>

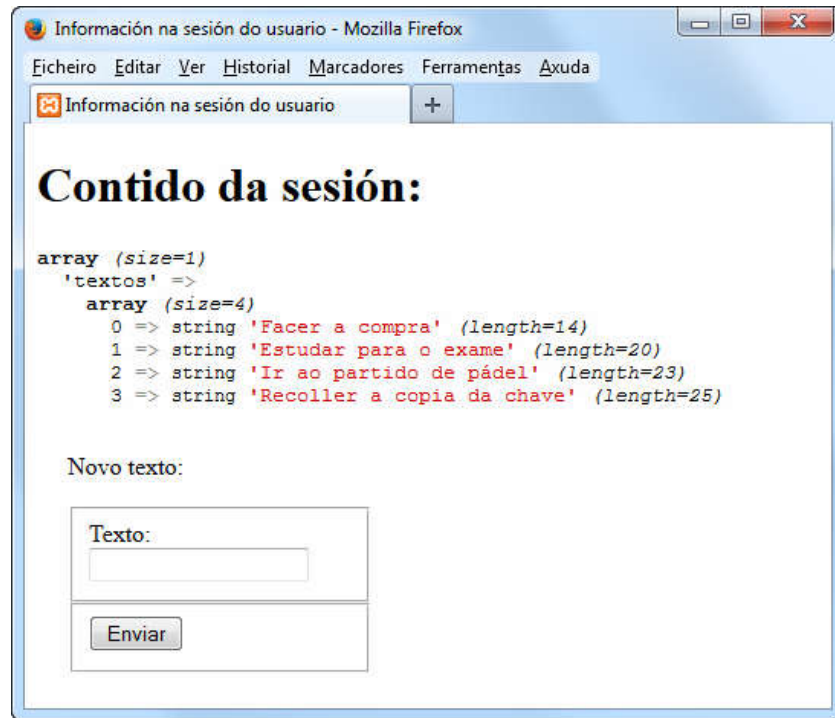
```

O código completo da páxina axúntase dentro do arquivo ZIP, na carpeta "tarefas/t1/t1b" co nome "cookies.php".

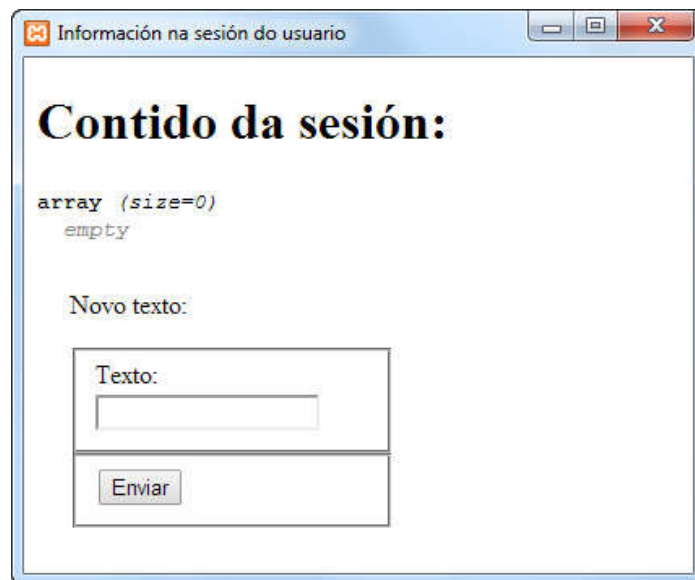
1.3.2 Tarefa 2. Almacenamento de información na sesión do usuario

a) Tarefa 2_a

Imos crear unha aplicación web que almacene unha serie de textos na sesión do usuario. Eses textos serán engadidos polo usuario empregando un pequeno formulario na parte inferior da páxina, semellante ao que fixemos nas tarefas do punto anterior.



Comprobar que a información se introduce de xeito correcto na sesión e que, ao cambiar de sesión (por exemplo, abrindo a páxina dende outro navegador) a lista de elementos aparece baleira pois trátase dunha sesión distinta.



b) Tarefa 2_b

Seguindo coa mesma páxina, e ao igual que fixemos na tarefa anterior, imos crear un novo botón que permita eliminar toda a información da sesión do usuario.



Autoavaliación

c) Tarefa 2_a

O código da páxina é moi semellante ao da tarefa anterior, pero nesta ocasión por exemplo non necesitamos cargar de novo a páxina para ver reflectidos os textos engadidos.

```
<?php
    session_start();

    if(!empty($_POST['texto'])) {
        $_SESSION['textos'][] = $_POST['texto'];
    }
?>

<!DOCTYPE html>

<html>

    <head>

        <title>Información na sesión do usuario</title>

        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">

        <style type="text/css">
            #novotexto {
                width: 200px;
                float: left;
                margin-left: 20px;
            }
            input[type=text] {
                width: 80%;
                display: block;
            }
        </style>

    </head>
```

```

<body>
    <h1>Contido da sesión:</h1>
    <?php var_dump($_SESSION); ?>
    <div id="novotexto">
        <form method='post'>
            <p>Novo texto:</p>
            <fieldset>
                <label for="texto">Texto:</label>
                <input id="texto" type="text" name="texto" />
            </fieldset>
            <fieldset>
                <input type="submit" value="Enviar" />
            </fieldset>
        </form>
    </div>
</body>
</html>

```

d) Tarefa 2_b

O código necesario para acadar o borrado dos datos da sesión do usuario cando se envía o formulario de borrado é o seguinte.

```

<?php
    session_start();
    if(!empty($_POST['texto'])) {
        $_SESSION['textos'][] = $_POST['texto'];
    }
    if(!empty($_POST['borrasesion'])) {
        session_unset();
        session_destroy();
    }
?>
<!DOCTYPE html>
<html>
    <head>
        <title>Información na sesión do usuario</title>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <style type="text/css">
            #novotexto {
                width: 200px;
                float: left;
                margin-left: 20px;
            }
            #borrasesion {
                width: 100px;
                float: left;

```

```

        margin: 60px 0 0 40px;
    }
    input[type=text] {
        width: 80%;
        display: block;
    }
</style>
</head>
<body>
    <h1>Contido da sesión:</h1>
    <?php var_dump($_SESSION); ?>
    <div id="novotexto">
        <form method='post'>
            <p>Novo texto:</p>
            <fieldset>
                <label for="texto">Texto:</label>
                <input id="texto" type="text" name="texto" />
            </fieldset>
            <fieldset>
                <input type="submit" value="Enviar" />
            </fieldset>
        </form>
    </div>
    <div id="borrasesion">
        <form method='post'>
            <fieldset>
                <input type="hidden" name="borrasesion" value="borra" />
                <input type="submit" value="Borrar a sesión" />
            </fieldset>
        </form>
    </div>
</body>
</html>

```

1.3.3 Tarefa 3. Autenticación de usuarios

a) Tarefa 3_a

Imos crear unha aplicación web que implemente un sistema sinxelo de rexistro de usuarios coas seguintes características:

- A aplicación comporase dunha única páxina.



- A lista de usuarios rexistrados na aplicación gardarase no servidor, nun arquivo de texto plano, `usuarios.txt`, que conterá para cada usuario dúas liñas:
 - A primeira liña correspóndese co nome do usuario.
 - A liña seguinte correspóndese co hash do seu contrasinal.

Por exemplo:

```
xaquin
$2y$10$amy81xf.PfXPUjtpBP.oreKSnVhUBQCCBA0kTkcvxxuK4DLDPK1FC
pepe
$2y$10$bt3EkV0cUDdh21W5FM0Ucu/LnvPY6hbZ/1fcokR7H9mbQyoDcD6Gy
carne
$2y$10$dEmjWD5/WLXfhFlRP9AbourYJ4sub.mjagIqZlaO.zEp8mlUtSJQ6
lola
$2y$10$Ra2oVUIbtaeIapF1Zv01Iu158lAdgwdaxiMrpieVoTVFBvzaPC2e.
xandre
$2y$10$GS3OwxIi7ucFKKyzurguxupiiQHo5eDnEEiQu4r5ZBeyMbXPXJFNe
```

- A aplicación permitirá rexistrarse a novos usuarios introducindo o seu nome de usuario e o seu contrasinal por duplicado (sección "Novo usuario"). Cando os contrasinais dun novo usuario coinciden, e o nome de usuario desexado non está en uso, engadirase unha nova liña coa súa información ao arquivo anterior.
- Os usuarios poderán loguearse na aplicación introducindo o seu nome de usuario e o seu contrasinal (sección "Usuario existente").
- A aplicación amosará en todo instante o nome do usuario logueado ou o texto "Non logueado" se é o caso.
- Cando haxa algún usuario logueado na aplicación, amosarase tamén un botón "Log out" que permita ao usuario saír. Neste caso pódese ocultar a sección que permita acceder a un usuario existente.

Login / Rexistro de usuarios - Mozilla Firefox

Ficheiro Editar Ver Historial Marcadores Ferramentas Axuda

Login / Rexistro de usuarios

Usuario xaquin logueado

Log out

Novo usuario:

Nome:

Contrasinal:

Repita a contrasinal:

Rexistrar

b) Tarefa 3_b

Seguindo coa mesma aplicación, imos engadir a posibilidade de que os usuarios indiquen cando se loguean un tempo máximo de inactividade, pasado o cal a aplicación os bota de xeito automático. Se o tempo indicado é 0, a sesión non caduca.

Login / Rexistro de usuarios - Mozilla Firefox

Ficheiro Editar Ver Historial Marcadores Ferramentas Axuda

Login / Rexistro de usuarios

Non logueado

Usuario existente:

Nome:

Contrasinal:

Segs. caducidade (0 = no caducar):

0

Log in

Novo usuario:

Nome:

Contrasinal:

Repita a contrasinal:

Rexistrar

Esta información deberá amosarse tamén na parte superior, indicando a hora na que vai caducar a sesión ou o texto "A sesión non caduca." se é o caso.

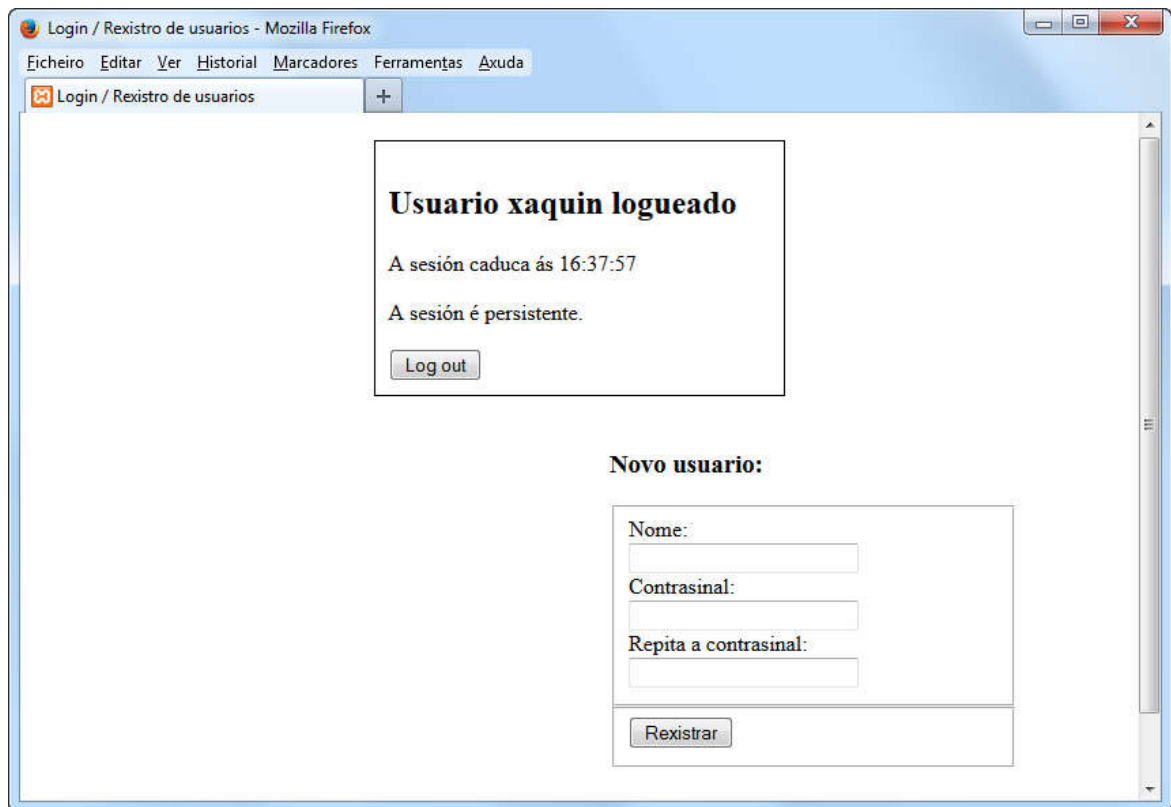
The screenshot shows a Mozilla Firefox browser window with the title 'Login / Rexistro de usuarios'. The address bar shows 'Login / Rexistro de usuarios'. The main content area displays a message box with the text 'Usuario xaquin logueado' and 'A sesión caduca ás 16:34:02'. Below this message is a 'Log out' button. To the right, under the heading 'Novo usuario:', there is a registration form with fields for 'Nome:', 'Contrasinal:', and 'Repita a contrasinal:', followed by a 'Rexistrar' button.

c) Tarefa 3_c

Para rematar, imos permitir tamén ao usuario indicar cando se loguea se desexa que a sesión que vai a abrir sexa permanente, isto é, que non se elimine cando se pecha o navegador e se poda recuperar nun prazo máximo de dúas semanas.

The screenshot shows the same Mozilla Firefox browser window. The main content area displays a message box with the text 'Non logueado'. Below this, there are two sections: 'Usuario existente:' and 'Novo usuario:'. The 'Usuario existente:' section contains a form with fields for 'Nome:', 'Contrasinal:', a checkbox labeled 'Mantenme rexistrado', a field for 'Segs. caducidade (0 = no caducar):' with the value '0', and a 'Log in' button. The 'Novo usuario:' section contains a form with fields for 'Nome:', 'Contrasinal:', 'Repita a contrasinal:', and a 'Rexistrar' button.

A información de se a sesión é ou non persistente tamén se deberá amosar xunto ao nome de usuario.



Recordade que Firefox e outros navegadores manteñen as *cookies* (e polo tanto os identificadores de sesión) daquelas lapelas que fican abertas cando se pecha o navegador; para forzar a eliminación das *cookies* permanentes deberemos pechar a lapela correspondente antes do navegador.

Autoavaliación

a) Tarefa 3_a

O código HTML da páxina "login.php" e as outras dúas páxinas de apoio que imos ver de seguido, axúntanse dentro do arquivo ZIP, na carpeta "tarefas/t3/t3a". Do seu contido cumpre destacar:

- Ao comezo de "login.php" débese chamar á función "session_start()".

```
<?php
    session_start();
    require_once('funcions.php');
```

- O nome do usuario gárdase nunha variable de sesión "nome_usuario". Cando esa variable existe, temos un usuario con ese nome logueado. Se a variable non existe, non temos ningún usuario logueado.

```
<div class="info">
    <?php
        if(empty($_SESSION['nome_usuario'])) echo '<h2>Non logueado</h2>';
        else {
            echo '<h2>Usuario ' . $_SESSION['nome_usuario'] . ' logueado</h2>';
        }
    ?>

    <form method='post' action='do_login.php'>
        <input type="hidden" name="logout" value="1" />
        <input type="submit" id="submit-logout" value="Log out" />
```

```

        </form>

<?php
    }
?>
</div>

```

- A páxina "login.php" inclúe tres formularios:
 - Un para permitir ao usuario saír do sistema, que soamente se amosa cando temos algún usuario logueado e contén soamente o botón de "Log out". Témoloo no código anterior.
 - Outro para permitir loguearse aos usuarios, que lles solicita o seu nome de usuario e o seu contrasinal.

```

<div class="form_login">
<?php if(!empty($_SESSION['nome_usuario'])) echo '
desactivado'; ?>
    <form method='post' action='do_login.php'>
        <h3>Usuario existente:</h3>
        <fieldset>
            <label for="nomeusuario">Nome:</label>
            <input id="nomeusuario" type="text" name="nomeusuario" />
            <label for="contrasinal">Contrasinal:</label>
            <input id="contrasinal" type="password" name="contrasinal" />
        </fieldset>
        <fieldset>
            <input type="submit" id="submit-login" value="Log in" />
        </fieldset>
    </form>
</div>

```

- Un terceiro para permitir rexistrarse a novos usuarios, que lles solicita o nome de usuario desexado e o contrasinal por duplicado.

```

<div class="form_registro">
    <form method='post' action='do_login.php'>
        <h3>Novo usuario:</h3>
        <fieldset>
            <label for="nomeusuario2">Nome:</label>
            <input id="nomeusuario2" type="text" name="nomeusuario" />
            <label for="novacontrasinal">Contrasinal:</label>
            <input id="novacontrasinal" type="password" name="contrasinal" />
            <label for="novacontrasinal2">Repita a contrasinal:</label>
            <input id="novacontrasinal2" type="password" name="contrasinal2" />
        </fieldset>
        <fieldset>
            <input type="submit" value="Rexistrar" />
        </fieldset>
    </form>

```

</div>

- Os tres formularios da páxina envían a información que recollen a unha nova páxina de nome "do_login.php" que en función dos datos recibidos:

- Elimina a sesión existente.

```
if(!empty($_POST['logout'])) {  
    // Log out  
    session_unset();  
    session_destroy();  
}
```

- Loguea a un usuario existente.

```
if(!empty($_POST['nomeusuario'])) {  
    // Log in  
    if(verifica_usuario($_POST['nomeusuario'], $_POST['contrasinal'])) {  
        $_SESSION['nome_usuario'] = $_POST['nomeusuario'];  
    }  
}
```

- Rexistra a un novo usuario.

```
if(!empty($_POST['contrasinal2'])) {  
    // Novo rexistro  
    garda_usuario($_POST['nomeusuario'],  
        $_POST['contrasinal'], $_POST['contrasinal2']);  
}
```

- Para facilitar a lexibilidade do código, creáronse algunhas funcións nun terceiro arquivo "funcions.php". Estas funcións empréganse dende a páxina "login.php".

- A función "hash_usuario" recupera do arquivo de texto o hash dun usuario a partir do seu nome. Se non existe o nome de usuario, devolve "false".

```
function hash_usuario($nome) {  
    @$f = fopen(FICH, 'r');  
    if($f) {  
        while(!feof($f)) {  
            $txt = trim(fgets($f));  
            if($nome == $txt) {  
                $hash = trim(fgets($f));  
                fclose($f);  
                return $hash;  
            }  
            fgets($f);  
        }  
    }  
    return false;  
}
```

- A función "garda_usuario" rexistra ao novo usuario se os seus datos son correctos.

```
function garda_usuario($nome, $contrasinal, $contrasinal2) {
    try {
        // Comprobamos que os contrasinais sexas iguais
        if($contrasinal !== $contrasinal2)
            throw new RuntimeException('Os contrasinais non coinciden');

        // Comprobamos que non exista xa ese nome de usuario
        if(hash_usuario($nome))
            throw new RuntimeException('O nome de usuario xa existe');
    }
    catch (RuntimeException $e) {
        echo $e->getMessage();
        exit();
    }

    // Rexistramos ao novo usuario
    $hash = password_hash($contrasinal, PASSWORD_DEFAULT);
    $texto = $nome . PHP_EOL . $hash . PHP_EOL;
    try {
        $f = fopen(FICH, 'a+');
        if(fwrite($f, $texto)) return true;
        else return false;
    }
    catch (RuntimeException $e) {
        echo $e->getMessage();
        exit();
    }
}
```

- A función "verifica_usuario" comproba se o nome de usuario e o contrasinal se corresponden aos dun usuario existente.

```
function verifica_usuario($nome, $contrasinal) {
    $hash_gardado = hash_usuario($nome);
    if($hash_gardado && password_verify($contrasinal, $hash_gardado))
        return true;
    else return false;
}
```

b) Tarefa 3_b

O código HTML das novas páxinas coas modificacións correspondentes axúntanse dentro do arquivo ZIP, na carpeta "tarefas/t3/t3b". Do seu contido cumpre destacar:

- Na páxina "login.php" engadimos:
 - Código ao comezo para comprobar se caducou a sesión, e para actualizar o tempo da última actividade do usuario.

```

if(!empty($_SESSION['nome_usuario'])) {
    // Comprobamos se caducou a sesión
    if(isset($_SESSION['ultima_actividade']) &&
        isset($_SESSION['tempo_caducidade']) &&
        (time() - $_SESSION['ultima_actividade'] >
            $_SESSION['tempo_caducidade'])) {
        session_unset();
        session_destroy();
    }
    $_SESSION['ultima_actividade'] = time();
}

```

– Código para amosar o estado da sesión:

```

<div class="info">
    <?php
        if(empty($_SESSION['nome_usuario'])) echo '<h2>Non logueado</h2>';
        else {
            echo '<h2>Usuario ' . $_SESSION['nome_usuario'] . ' logueado</h2>';
            if(isset($_SESSION['tempo_caducidade']))
                echo '<p>A sesión caduca ás ' . date('H:i:s', time() +
                    $_SESSION['tempo_caducidade']) . '</p>';
            else
                echo '<p>A sesión non caduca.</p>';
        }
    ?>
    <form method='post' action='do_login.php'>
        <input type="hidden" name="logout" value="1" />
        <input type="submit" id="submit-logout" value="Log out" />
    </form>
    <?php
        }
    ?>
</div>

```

– Un campo ao formulario para que o usuario decida o tempo de caducidade da sesión:

```

<fieldset>
<label for="tempo_caducidade">Segs. caducidade (0 = no caducar):</label>
    <input id="tempo_caducidade" type="text" name="tempo_caducidade" value="0"
    />
</fieldset>

```

- Na páxina "do_login.php" engadimos código para que, cando se loguea un usuario, se garde na sesión o tempo de caducidade:

```

if(verifica_usuario($_POST['nomeusuario'], $_POST['contrasinal'])) {
    $_SESSION['nome_usuario'] = $_POST['nomeusuario'];
}

```

```
// Comprobamos se se definiu un tempo de caducidade da sesión
if(!empty($_POST['tempo_caducidade'])) {
    if($_POST['tempo_caducidade'] > 0)
        $_SESSION['tempo_caducidade'] = $_POST['tempo_caducidade'];
    else
        unset($_SESSION['tempo_caducidade']);
}
}
```

c) Tarefa 3_c

O código HTML das novas páxinas coas modificacións correspondentes axúntanse dentro do arquivo ZIP, na carpeta "tarefas/t3/t3c". Do seu contido cumpre destacar:

- Na páxina "login.php" engadimos:
 - Código para que, no caso de que non teñamos a ningún usuario logueado, comprobar se existen as *cookies* correspondentes a unha sesión anterior, e restaurala.

```
<?php
    session_start();
    require_once('funcions.php');

    if(empty($_SESSION['nome_usuario'])) {
        // Comprobamos se existen as cookies para recuperar unha sesión anterior
        if(isset($_COOKIE['login']) && isset($_COOKIE['hash']))
            recupera_sesion($_COOKIE['login'], $_COOKIE['hash']);
    }
```

- As etiquetas HTML necesarias para amosar se a sesión é ou non persistente.

```
<div class="info">
    <?php
        if(empty($_SESSION['nome_usuario'])) echo '<h2>Non logueado</h2>';
        else {
            echo '<h2>Usuario ' . $_SESSION['nome_usuario'] . ' logueado</h2>';
            if(isset($_SESSION['tempo_caducidade']))
                echo '<p>A sesión caduca ás ' . date('H:i:s', time() +
                    $_SESSION['tempo_caducidade']) . '</p>';
            else
                echo '<p>A sesión non caduca.</p>';
            if(isset($_SESSION['manter_rexistrado']) &&
                $_SESSION['manter_rexistrado']==true)
                echo '<p>A sesión é persistente.</p>';
            else
                echo '<p>A sesión non é persistente.</p>'
        }
    ?>
```

- O novo campo no formulario no que se loguean os usuarios:

```
<fieldset>
```

```

        <input type="checkbox" checked="checked" id="manter_rexistrado"
name="manter_rexistrado" /><label> Mantenme rexistrado</label>
    </fieldset>
?>

```

- Iremos empregar unha nova función "recupera_sesion" na páxina "funcions.php", para restaurar a información dunha sesión a partir do nome de usuario e do *hash* almacenado nas *cookies* do navegador.

```

function recupera_sesion($nome, $hash) {
    global $salt;

    $texto = $nome.$_SERVER['HTTP_USER_AGENT'].$_SERVER['REMOTE_ADDR'];

    if($hash == crypt($texto, $salt)) {
        // Recuperamos a sesión do usuario
        $_SESSION['nome_usuario'] = $nome;
        $_SESSION['manter_rexistrado'] = true;

        // Se tiveramos gardada algunha outra información
        // como o tempo de caducidade, deberiamos recuperala neste intre
    }
}

```

- E para rematar, na páxina "do_login.php" temos que facer un par de cambios.
 - Cando un usuario se loguea, no caso de solicitar unha sesión permanente deberemos crear as *cookies* correspondentes no navegador.

```

// Log in
if(verifica_usuario($_POST['nomeusuario'], $_POST['contrasinal'])) {
    $_SESSION['nome_usuario'] = $_POST['nomeusuario'];

    // Comprobamos se se definiu un tempo de caducidade da sesión
    if(!empty($_POST['tempo_caducidade'])) {
        if($_POST['tempo_caducidade'] > 0)
            $_SESSION['tempo_caducidade'] = $_POST['tempo_caducidade'];
        else
            unset($_SESSION['tempo_caducidade']);
    }

    // Comprobamos se se definiu a creación dunha sesión permanente
    if(!empty($_POST['manter_rexistrado']) &&
        $_POST['manter_rexistrado'] == true) {
        $texto = $_SESSION['nome_usuario'] . $_SERVER['HTTP_USER_AGENT'] .
            $_SERVER['REMOTE_ADDR'];

        setcookie("login", $_SESSION['nome_usuario'], time()+3600*24*14);
        setcookie("hash", crypt($texto, $salt), time()+3600*24*14);
    }
}

```

```
$_SESSION['manter_rexistrado'] = $_POST['manter_rexistrado'];  
}
```

- Cando un usuario sae da aplicación, deberemos borrar tamén as *cookies* necesarias para manter a sesión permanente, se existen. De non facelo, os usuarios non serían quen de saír da aplicación.

```
if(!empty($_POST['logout'])) {  
    // Log out  
    session_unset();  
    session_destroy();  
  
    // Quitamos as cookies para a sesión permanente, se existen  
    setcookie("login", '', time() - 3600);  
    setcookie("hash", '', time() - 3600);  
}
```

2. Materiais

2.1 Textos de apoio ou de referencia

- **Programming PHP, 3rd Ed**, Kevin Tatroe, Peter MacIntyre e Rasmus Lerdorf, Ed. O'Reilly.
- **Beginning PHP 5.3**, Matt Doyle, Ed. Wrox.
- **Pro PHP Security, From Application Security Principles to the Implementation of XSS Defenses**, Chris Snyder, Thomas Myer e Michael Southwell, Ed. Apress.
- <http://www.php.net/>, documentación sobre a linguaxe PHP.
- <http://www.w3schools.com/php/default.asp>, documentación sobre a linguaxe PHP.
- TA1.

2.2 Recursos didácticos

- Ordenador persoal con:
 - Software de servidor web Apache ou semellante.
 - Intérprete da linguaxe PHP.
 - Contorno de desenvolvemento de aplicacións web NetBeans ou semellante.
 - Navegador web Firefox ou semellante.
- Recomendado tamén ter conexión a Internet.

3. TA1. Cálculo do hash en PHP

Vexamos as opcións que temos en PHP para calcular o *hash*, pensando principalmente no seu emprego cos contrasinais dos usuarios dunha aplicación web.

3.1 Funcións e extensións para o cálculo do hash

Na librería de funcións de PHP temos as seguintes opcións que nos permiten obter unha cadea *hash* dun texto:

- **Empregar a función `md5()`**, que devolve o *hash* MD5 dun texto. A función `md5_file()` calcula o mesmo *hash* sobre o contido do arquivo que se lle indica.

```
$salt = 'GKM")OI4%MGfd".g';
$contrasinal = 'contrasinal';
echo 'MD5: ' . md5($salt.$contrasinal);
// MD5: 1a0d7c12f1feb1fae8c2b734638fe6fb
```

- **Empregar a función `sha1()`**, que devolve o *hash* SHA1 dun texto. A función `sha1_file()` calcula o mesmo *hash* sobre o contido do arquivo que se lle indica.

```
echo 'SHA-1: ' . sha1($salt.$contrasinal);
// SHA-1: 09f34f8390b858c282297cf9670c988da8581d97
```

- **Empregar a función `crypt()`**, que aplica un algoritmo de *hash* sobre o texto e o *salt* que se lle pasan (en parámetros separados) e devolve o resultado. O algoritmo empregado escóllese entre os seguintes en función do *salt* e da súa dispoñibilidade no sistema operativo: DES, MD5, Blowfish, SHA-256 ou SHA-512. Así, a función `crypt` intenta empregar o algoritmo:

- **DES estándar** cando o *salt* está composto por 2 dos seguintes caracteres: `"/0-9A-Za-z"`.

```
echo 'DES Estándar: ' . crypt($contrasinal, '7J');
// DES Estándar: 7JC/VdPVB0pg2
```

- **DES estendido** cando o *salt* comeza polo carácter `"_"` e a continuación contén 4 caracteres para indicar o número de iteracións e outros 4 para o *salt* propiamente dito. Estes oito caracteres deben ser do conxunto `"/0-9A-Za-z"`.

```
echo 'DES Estendido: ' . crypt($contrasinal, '_AA..5T0K');
// DES Estendido: _AA..5T0KE0q8CWN4tpg
```

- **MD5** cando o *salt* contén doce caracteres comezando por `"$1$"`.

```
echo 'MD5: ' . crypt($contrasinal, '$1$6YK/09.sd2');
// MD5: $1$6YK/09.s$SwmnpM9GPlQFiQl1LUu920
```

- **Blowfish** cando o *salt* comeza por `"$2a$"`, `"$2x$"` ou `"$2y$"`. A continuación debe figurar o número de iteracións, entre 04 e 31, un signo `"$"`, e 22 caracteres do alfabeto `"/0-9A-Za-z"`. Nas versións de PHP 5.3.7 e superiores, debería escollerse `"$2y$"` como cadea inicial.

```
echo 'Blowfish: ' . crypt($contrasinal, '$2y$08$j/2Do/3dGF..Ep9s//dA4B');
```

```
// Blowfish: $2y$08$j/2Do/3dGF..Ep9s//dA4.BvtbJWNkQX0xz4zmiHpZtdXnPFUc61W
```

- **SHA-256** cando o *salt* comeza por "\$5\$" e contén outros 16 caracteres.

```
echo 'SHA256: ' . crypt($contrasinal, '$5$GKM')OI4%MGfd".g');  
// SHA256: $5$GKM")OI4%MGfd".g$izVoWp0CrIwSFVr6tuHUIQkosFWGBluPPDv.q.syEP8
```

- **SHA-512** cando o *salt* comeza por "\$6\$" e contén outros 16 caracteres. Tanto neste caso como no anterior, pódese engadir unha cadea co formato "rounds=N\$" antes destes caracteres para indicar o número de iteracións, onde $1000 \leq N < 1.000.000.000$.

```
echo 'SHA512: ' . crypt($contrasinal, '$6$rounds=5000$GKM')OI4%MGfd".g');  
// SHA512: $6$rounds=5000$GKM")OI4%MGfd".g$Zkx4X7mV1oVOdtY/fpEemyC7NRw/yT6  
ocgL932f5fgZwCAmkXTAFitf2En2NBt5qit6UPJHuglPcpJJZynP7C0
```

Como se pode ver polos resultados obtidos, a función `crypt` engade sempre o *salt* empregado ao comezo do *hash* obtido. Isto faise para non ter que almacenar o *salt* na nosa aplicación, e permite empregar un *salt* distinto con cada contrasinal. Deste xeito, e dado que a función `crypt` omite os caracteres sobrantes do *salt* se existe, podemos empregar o seguinte método para comprobar os contrasinais dos usuarios:

```
$salt = '(cadea de salt válida para o algoritmo que empreguemos)';  
$hash = crypt($contrasinal, $salt);  
  
// Almacenamos o hash do usuario  
  
// A seguinte vez que inicie sesión o usuario, comprobamos o seguinte  
if ($hash == crypt($contrasinal, $hash)) {  
    // Contrasinal correcta  
}
```

- **Empregar a extensión `hash`**, que forma parte do núcleo de PHP. A extensión soporte múltiples algoritmos de *hash*, como MD5, ou distintas variantes de SHA e de RIPEMD. Podemos empregar a función `hash_algos()` para ver a lista completa. O algoritmo que imos a empregar débese especificar como primeiro parámetro na chamada á función `hash()`.

```
$salt = 'GKM")OI4%MGfd".g';  
$contrasinal = 'contrasinal';  
  
echo 'RIPEMD160: ' . hash('ripemd160', $salt.$contrasinal) . "<br />";  
// RIPEMD160: a91367d6c8963c452f23b82af57c92af07a59b5c
```

- **Empregar a extensión `mcrypt`**, que non ven incluída normalmente con PHP e debémosa instalar con posterioridade.

```
sudo apt-get install php5-mcrypt  
sudo /etc/init.d/apache2 restart
```

`mcrypt` soporta moitos algoritmos de *hash* como DES, Triple DES ou Blowfish.

- **Empregar a extensión *Password hashing***, que forma parte do núcleo de PHP dende a versión 5.5.0. É un xeito máis sinxelo de facer chamadas a función `crypt` orientado ao seu emprego con contrasinais. Baséase en dúas funcións:

- `password_hash()`. Crea un *hash* a partir dun texto e unha constante que indique o algoritmo a empregar. Temos dúas opcións:
 - `PASSWORD_BCRYPT`. Emprega o algoritmo "bcrypt", unha variante de Blowfish.
 - `PASSWORD_DEFAULT`. Emprega o algoritmo que estea considerado como máis seguro e pode variar con cada versión de PHP. Agora mesmo o algoritmo empregado é "bcrypt".

```
$hash = password_hash($contrasinal, PASSWORD_DEFAULT);
echo 'Password hash: ' . $hash . "<br />";

// Password hash:
$2y$10$WG94Mxlvd.oWK5D0MDh4DekuIqMnbwttim1CJfK3iNjVNNrqcRc46
```

- `password_verify()`. Recibe como parámetros o contrasinal e o *hash* almacenado, e indica se o contrasinal é correcto (`true`) ou non (`false`).

```
if (password_verify($contrasinal, $hash)) echo 'O contrasinal é válido!';
else echo 'O contrasinal é inválido!';
```

Dado que o algoritmo a empregar pola extensión "*Password hashing*" e as súas opcións poden variar con distintas chamadas as funcións da extensión, esta información engádense ao *hash* obtido pola función `password_hash`. A función `password_verify` emprega esta información para facer a comprobación dos contrasinais.

A elección dun algoritmo de *hash* depende en gran medida do seu obxectivo. Por exemplo, un algoritmo de *hash* para poder comprobar que un ficheiro se descargou ou copiou sen modificarse debe ser rápido. Pola contra, un algoritmo *hash* para aplicar aos contrasinais debe ter certa complexidade computacional, de xeito que se un atacante faise co *hash* dun contrasinal non sexa sinxelo empregar ataques de forza bruta para obter o seu contrasinal asociado.

As funcións `md5()` e `sha1()` foron ata hai pouco un estándar, pero pola rapidez na extracción de datos por forza bruta, diccionario ou táboas "*rainbow*" xa non se recomendan para a codificación de contrasinais. No seu lugar, recoméndase empregar o algoritmo Blowfish ou, aínda mellor, a extensión "*Password hashing*".